Unit-II

2.4 Nested query in SQL

A nested query in SQL, also known as a subquery or inner query, is a complete SQL query embedded within another SQL query (the outer query). The result of the inner query is used by the outer query to achieve a specific data retrieval or manipulation goal.

Key characteristics and uses of nested queries:

- **Execution Order:**

  The inner query executes first, and its result set is then used as input or a condition for the outer query.

- **Placement:**

  Nested queries are commonly found within the WHERE, FROM, or HAVING clauses of the outer query, and can also be used in SELECT, INSERT, UPDATE, or DELETE statements.

- **Types of Results:**

  A subquery can return a single value (scalar), a single row, a single column, or a complete table.

- **Common Operators:**

  Operators like IN, NOT IN, ANY, ALL, EXISTS, and comparison operators (=, >, <) are frequently used in conjunction with subqueries in the WHERE clause.

- **Purpose:**

  Nested queries are useful for:

- Filtering data based on complex conditions that require a preliminary query.
- Performing calculations or aggregations on a subset of data before using the result in the outer query.
- Indirectly joining datasets without explicitly using JOIN clauses, especially when complex relationships or filtering are involved.
- Breaking down complex problems into smaller, more manageable parts.

Example:

To find the names of employees who earn more than the average salary of all employees:

Code

```
SELECT employee_name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

In this example, the inner query (SELECT AVG(salary) FROM employees) calculates the average salary, and this single value is then used by the outer query to filter employees whose salaries exceed that average.

SQL Nested Queries

A nested query (also called a subquery) is a query embedded within another SQL query. The result of the inner query is used by the outer query to perform additional operations. Subqueries can be used in various parts of an SQL query such as SELECT, FROM or WHERE Clauses.

- The inner query runs first, providing data for the outer query.
- The output query uses the results of the inner query for **comparison** or as input.
- Nested queries are particularly useful for breaking down complex problems into smaller, manageable parts, making it easier to retrieve specific results from large datasets.

Why Use Nested Queries?

Nested queries are highly useful when:

- We need to retrieve data based on the results of another query.
- We want to perform filtering or aggregation without using joins.
- We need to break down complex operations into smaller steps.

To better understand nested queries, we will use the following sample tables: **STUDENT, COURSE,** and **STUDENT_COURSE**. These tables demonstrate a real-world scenario of students, courses and their enrollment details, which will be used in the examples below.

1. STUDENT Table

The **STUDENT** table stores information about students, including their unique ID, name, address, phone number, and age.

S_ID	S_NAME	S_ADDRESS	S_PHONE	S_AGE
S1	RAM	DELHI	9455123451	18
S2	RAMESH	GURGAON	9652431543	18
S3	SUJIT	ROHTAK	9156253131	20
S4	SURESH	DELHI	9156768971	18

2. COURSE Table

The **STUDENT_COURSE** table maps students to the courses they have enrolled in. It uses the student and course IDs as foreign keys.

C_ID	C_NAME
C1	DSA
C2	Programming
C3	DBMS

3. STUDENT_COURSE Table

This table maps students to the courses they have enrolled in, with columns for student ID (**S_ID**) and course ID (**C_ID**):

S_ID	C_ID
S1	C1
S1	C3
S2	C1
S3	C2
S4	C2
S4	C3

Types of Nested Queries in SQL

There are two primary types of nested queries in SQL, Independent Nested Queries and Correlated Nested Queries. Each type has its own use case and benefits depending on the complexity of the task at hand.

1. Independent Nested Queries

In an independent nested query, the execution of the inner query is independent of the outer query. The inner query runs first and its result is used directly by the outer query.

Operators like **IN**, **NOT IN**, **ANY** and **ALL** are commonly used with independent nested query.

Example 1: Using IN

In this Example we will find the S_IDs of students who are enrolled in the courses '**DSA**' or '**DBMS**'. We can break the query into two parts:

Step 1: Find the C_IDs of the courses:

This query retrieves the IDs of the courses named 'DSA' or 'DBMS' from the **COURSE** table.

SELECT C_ID FROM COURSE WHERE C_NAME IN ('DSA', 'DBMS');

Output

C_ID
C1
C3

Step 2: Use the result of Step 1 to find the corresponding S_IDs:

The inner query finds the course IDs, and the outer query retrieves the student IDs associated with those courses from the STUDENT_COURSE table

SELECT S_ID FROM STUDENT_COURSE WHERE C_ID IN (
SELECT C_ID FROM COURSE WHERE C_NAME IN ('DSA', 'DBMS')
);

Output

S_ID
S1
S2

S_ID
S4

Explanation: In this example, the inner query retrieves the C_IDs of the courses 'DSA' and 'DBMS', and the outer query retrieves the student IDs (S_IDs) enrolled in those courses.

2. Correlated Nested Queries

In correlated nested queries, the inner query depends on the outer query for its execution. For each row processed by the outer query, the inner query is executed. This means the inner query references columns from the outer query. The EXISTS keyword is often used with correlated queries.

Example 2: Using EXISTS

In this Example, we will find the names of students who are enrolled in the course with **C_ID = 'C1'**:

```
SELECT S_NAME FROM STUDENT S
WHERE EXISTS (
SELECT 1 FROM STUDENT_COURSE SC
WHERE S.S_ID = SC.S_ID AND SC.C_ID = 'C1'
);
```

Output

S_NAME
RAM
RAMESH

Explanation:

For each student in the STUDENT table, the inner query checks if an entry exists in the STUDENT_COURSE table with the same S_ID and the specified C_ID. If such a record exists, the student's name is included in the output.

Common SQL Operators for Nested Queries

SQL provides several operators that can be used with nested queries to filter, compare, and perform conditional checks.

1. IN Operator

The IN operator is used to check whether a column value matches any value in a list of values returned by a subquery. This operator simplifies queries by avoiding the need for multiple OR conditions.

Example: Retrieve student names who enrolled in 'DSA' or 'DBMS':

This query filters the students enrolled in the specified courses by chaining multiple nested queries.

```
SELECT S_NAME FROM STUDENT
WHERE S_ID IN (
SELECT S_ID FROM STUDENT_COURSE
WHERE C_ID IN (
SELECT C_ID FROM COURSE WHERE C_NAME IN ('DSA', 'DBMS')
)
);
```

2. NOT IN Operator

The NOT IN operator excludes rows based on a set of values from a subquery. It is particularly useful for filtering out unwanted results. This operator helps identify records that do not match the conditions defined in the subquery.

Example: Retrieve student IDs not enrolled in 'DSA' or 'DBMS':

This query excludes students who are enrolled in the courses 'DSA' or 'DBMS'.

```
SELECT S_ID FROM STUDENT
WHERE S_ID NOT IN (
SELECT S_ID FROM STUDENT_COURSE
WHERE C_ID IN (
SELECT C_ID FROM COURSE WHERE C_NAME IN ('DSA', 'DBMS')
)
);
```

Output

S_ID
S3

3. EXISTS Operator

The EXISTS operator checks for the existence of rows in a subquery. It returns true if the subquery produces any rows, making it efficient for conditional checks. This operator is often used to test for relationships between tables.

Example: Find student names enrolled in 'DSA'

The inner query checks for matching records in the STUDENT_COURSE table, and the outer query returns the corresponding student names.

```
SELECT S_NAME FROM STUDENT S
WHERE EXISTS (
SELECT 1 FROM STUDENT_COURSE SC
WHERE S.S_ID = SC.S_ID AND SC.C_ID = 'C1'
);
```

4. ANY and ALL Operators

- ANY: Compares a value with **any value** returned by the subquery.
- ALL: Compares a value with **all values** returned by the subquery.

Example using ANY:

```
SELECT S_NAME FROM STUDENT
WHERE S_AGE > ANY (
SELECT S_AGE FROM STUDENT WHERE S_ADDRESS = 'DELHI'
);
```

Example using ALL:

```
SELECT S_NAME FROM STUDENT
WHERE S_AGE > ALL (
SELECT S_AGE FROM STUDENT WHERE S_ADDRESS = 'DELHI'
);
```

Advantages of Nested Queries

- **Simplifies complex queries:** Nested queries allow us to divide complicated SQL tasks into smaller, more manageable parts. This modular approach makes queries easier to write, debug, and maintain.
- **Enhances flexibility:** By enabling the use of results from one query within another, nested queries allow dynamic filtering and indirect joins, which can simplify query logic.
- **Supports advanced analysis:** Nested queries empower developers to perform operations like conditional aggregation, subsetting, and customized calculations, making them ideal for sophisticated data analysis tasks.
- **Improves readability:** When properly written, nested queries can make complex operations more intuitive by encapsulating logic within inner queries.